

PE-EC802C — VLSI Design Automation

Semester VIII • 3L:0T:0P • 3 Credits

Module 4: VLSI Simulation & Logic Synthesis

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	VLSI Simulation Techniques	2
1.1	Gate-Level Event-Driven Simulation	2
1.2	Switch-Level Simulation	2
1.3	Compiled-Code vs. Event-Driven Simulation	2
2	Combinational Logic Synthesis	3
2.1	1. Technology-Independent Optimization	3
2.2	2. Technology Mapping	4
3	Two-Level Logic Synthesis	4
3.1	Quine-McCluskey (QM) Minimisation	4
3.2	Espresso Heuristic Minimiser	4
4	Binary Decision Diagrams (BDD)	5
4.1	Shannon’s Expansion Theorem	5
4.2	Reduced Ordered BDDs (ROBDD)	5
4.3	The BDD Apply Operation	6

1. VLSI Simulation Techniques

VLSI Simulation

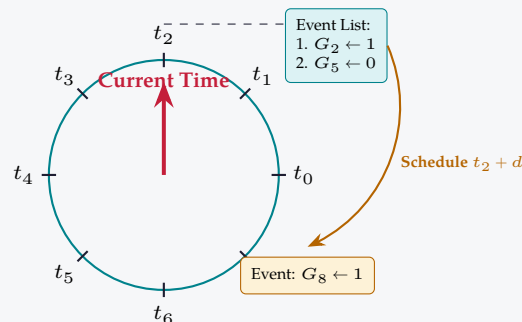
VLSI Simulation is the process of building a software model of a digital circuit and executing it with test input sequences to verify its functional correctness, check signal timings, and isolate design faults prior to physical fabrication.

1.1 Gate-Level Event-Driven Simulation

Event-driven simulators calculate logic outputs only when an input signal transition (an *event*) occurs. This is highly efficient because less than 10% of gates in a typical digital circuit switch states during any clock cycle.

- **Event Queue:** Stores pending signal updates sorted by execution time.
- **Time Wheel:** A circular queue scheduling events at future times. A pointer represents current simulation time (t). When the pointer moves to slot i , the events linked to t_i are evaluated. If a gate output changes, new events are scheduled at $t_i + d$ (where d is gate delay).

Time Wheel Scheduler Concept



1.2 Switch-Level Simulation

Models digital circuits at the transistor level by representing MOS transistors as voltage-controlled switches.

- **States:** Signals are modeled using more than two states, usually including 0 (low), 1 (high), X (unknown), and Z (high impedance).
- **Strengths:** Signals are assigned strengths to resolve bus contention (e.g. supply strength, pull strength, charge storage strength). An active pull-up transistor overrides a weak pull-down resistor.

1.3 Compiled-Code vs. Event-Driven Simulation

Gate-level simulators evaluate logic gates using two primary simulation paradigms:

- **Compiled-Code Simulation:** The digital circuit is translated into a sequence of machine instructions that are executed line-by-line. It evaluates the logic value of every gate in

strict topological order during every simulation cycle, regardless of whether its input values have changed. It is highly efficient for circuits with very high switching activity, but extremely wasteful for typical circuits where only a small fraction ($< 10\%$) of gate inputs change state at any step.

- **Event-Driven Simulation:** Only evaluates a logic gate when a transition (event) occurs on at least one of its inputs. It schedules events dynamically using a priority queue (event queue or time wheel). While it avoids redundant gate evaluations, it incurs runtime overhead for queue management. It is the preferred method for large digital systems with low-to-medium active switching.

Feature	Compiled-Code Simulation	Event-Driven Simulation
Evaluation Rule	Evaluates all gates in every simulation cycle.	Evaluates gates only when input signals switch states.
Overhead	Zero scheduling or queue management overhead.	Dynamic scheduling and queue sorting overhead.
Optimal Activity	Best suited for high switching activity circuits.	Best suited for low-to-medium switching activity ($< 10\%$).
Delay Modeling	Strictly cycle-based; difficult to model gate delays.	Models detailed gate propagation and inertial delays.

2. Combinational Logic Synthesis

Logic synthesis converts high-level RTL descriptions (Verilog/VHDL) into an optimized gate-level netlist. The combinational logic synthesis process consists of two primary sequential phases:

2.1 1. Technology-Independent Optimization

Optimizes the Boolean behavior of the network without considering specific library gates. The circuit is modeled as a **Boolean Network** (a Directed Acyclic Graph where nodes represent Boolean functions and edges represent signal flow).

- **Restructuring Operations:**
 - **Decomposition:** Splitting a complex node function into smaller, simpler functions.
 - **Extraction:** Identifying common sub-expressions across multiple nodes and creating a new node to share them, reducing overall literal count.
 - **Factoring:** Converting SOP forms to factored forms (e.g. $ab+ac \rightarrow a(b+c)$) to minimize transistor count.
 - **Substitution:** Expressing a node's function in terms of other existing nodes.
 - **Collapsing:** Merging a node into its fan-out nodes to eliminate intermediate nodes, reducing delay at the expense of area.
- **And-Inverter Graphs (AIGs):** A representation where every node is a 2-input AND gate, and edges can contain inversion flags. Highly efficient for Boolean reasoning and logic optimization.

2.2 Technology Mapping

Translates the optimized technology-independent Boolean network into standard library cells of the target fabrication process (e.g. static CMOS NAND, NOR, AOI gates).

- **Tree Matching (Dagon's Algorithm):**
 - The Boolean network is partitioned into a collection of trees.
 - Library gates are also represented as patterns of trees.
 - Dynamic programming is applied to find the minimum-cost cover of standard cells that matches the network tree structures.
- **Objectives:** Optimizing for minimum area, minimum delay (critical path), or minimum power dissipation.

3. Two-Level Logic Synthesis

Logic synthesis compiles high-level RTL descriptions (Verilog/VHDL) into an optimized gate-level netlist. Two-level logic synthesis targets sum-of-products (SOP) forms.

3.1 Quine-McCluskey (QM) Minimisation

An exact tabular method that guarantees finding the absolute minimum SOP expression.

1. Represent minterms in binary and group them by the number of 1s they contain.
2. Combine minterms that differ by exactly one bit (e.g., 0100 and 0101 combine to 010—), replacing the differing bit with a dash.
3. Repeat combinations until no more terms can be merged. The remaining uncombined terms are **Prime Implicants (PI)**.
4. Construct a Prime Implicant chart, identify **Essential Prime Implicants (EPI)** (columns containing only one checkmark), and select the minimum subset of PIs to cover the remaining minterms.

3.2 Espresso Heuristic Minimiser

Because QM suffers from exponential time complexity ($O(2^{2^n})$) and is unusable for more than 10 inputs, industrial synthesis tools use the Espresso heuristic optimizer. Key iterative steps in Espresso:

- **Expand:** Squeezes out variables by expanding cubes to cover adjacent minterms, reducing the total number of terms.
- **Irredundant:** Selects a minimum subset of prime cubes that covers the entire ON-set, deleting redundant terms.
- **Reduce:** Shrinks cubes to their smallest possible dimensions without uncovering any minterms, allowing the algorithm to search in new directions during the next Expand phase.

4. Binary Decision Diagrams (BDD)

Binary Decision Diagram

A **Binary Decision Diagram (BDD)** is a directed acyclic graph (DAG) representation of a Boolean function. It is constructed by recursively applying Shannon's expansion theorem.

4.1 Shannon's Expansion Theorem

Any Boolean function $f(x_1, x_2, \dots, x_n)$ can be expanded about variable x_1 as:

$$f = x_1 \cdot f_{x_1} + x_1' \cdot f_{x_1'} \quad (1)$$

where $f_{x_1} = f(1, x_2, \dots, x_n)$ is the positive cofactor, and $f_{x_1'} = f(0, x_2, \dots, x_n)$ is the negative cofactor.

Shannon's Expansion Example

Problem:

Expand the function $f = AB + B'C$ about variable A .

Solution:

1. Compute cofactors about A :

$$\begin{aligned} f_A &= (1)B + B'C = B + C \quad (\text{using absorption } B + B'C = B + C) \\ f_{A'} &= (0)B + B'C = B'C \end{aligned}$$

2. Combine terms using Shannon's theorem:

$$f = A \cdot (B + C) + A' \cdot (B'C)$$

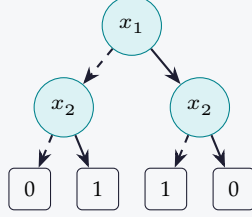
4.2 Reduced Ordered BDDs (ROBDD)

An **Ordered BDD (OBDD)** is a BDD where variables are evaluated in a fixed order along every path from root to leaf. A **Reduced OBDD (ROBDD)** is obtained by applying three reduction rules:

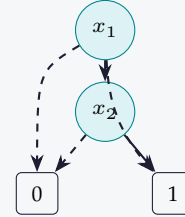
1. **Merge Duplicate Leaves:** Combine all leaf nodes containing 0 into a single 0-terminal, and all 1-leaves into a single 1-terminal.
2. **Merge Duplicate Nodes:** If two internal nodes u and v represent the same variable and have identical low and high child nodes, merge them.
3. **Eliminate Redundant Nodes:** If an internal node has the same low and high child node w , delete the node and redirect all incoming edges directly to w .

BDD to ROBDD Reduction Steps

Unreduced BDD:



Reduced ROBDD (after merging):



ROBDDs are canonical representations of Boolean functions. For a fixed variable ordering, every Boolean function has exactly one unique ROBDD.

4.3 The BDD Apply Operation

The BDD **Apply** algorithm is the fundamental operation used to combine two BDDs with a binary Boolean operator (such as AND, OR, XOR). Given BDDs for f and g , the algorithm computes the BDD for $h = f \text{ op } g$ in $O(|f| \cdot |g|)$ polynomial time.

Recursive Algorithm Principle (Shannon Expansion): Let x be the variable with the highest priority (lowest index in the variable ordering) among the root variables of BDDs f and g . We can expand $f \text{ op } g$ about x as:

$$f \text{ op } g = x \cdot (f_x \text{ op } g_x) + x' \cdot (f_{x'} \text{ op } g_{x'}) \quad (2)$$

where f_x and $f_{x'}$ are cofactors of f with respect to x ($f_x = f(1)$ and $f_{x'} = f(0)$). If f does not depend on x , then $f_x = f_{x'} = f$.

Algorithmic Flow:

1. **Base Cases:** If f and g are terminal nodes (0 or 1), evaluate the operation directly (e.g., $1 \text{ AND } 0 = 0$).
2. **Memoization (Computed Table):** Before recursively expanding, check if the triple (f, g, op) has already been computed. If yes, return the cached result. This avoids exponential recursive redundancy.
3. **Recursion:** Compute the cofactors, and recursively call the Apply algorithm on the positive cofactors $h_{\text{high}} = \text{Apply}(\text{op}, f_x, g_x)$ and negative cofactors $h_{\text{low}} = \text{Apply}(\text{op}, f_{x'}, g_{x'})$.
4. **Reduction:** If $h_{\text{high}} == h_{\text{low}}$, return h_{low} (redundant node elimination). Otherwise, find or create a node for variable x pointing to h_{high} (as high child) and h_{low} (as low child), and store the result in the computed table.

Quick Revision — Module 4

Logic Synthesis, Simulation & BDDs | PE-EC802C

Key Formulas and Metrics at a Glance

Shannon's Expansion	$f = x_i \cdot f_{x_i} + x'_i \cdot f_{x'_i}$ (foundation for BDD node building)
QM Worst-Case Implicants	$N_{PI} \approx \frac{3^n}{n}$ (for n variables)
Event Scheduler Delay	$t_{\text{schedule}} = t_{\text{current}} + d_{\text{gate}}$ (event propagation delay)

One-Line Definitions

- **Time Wheel:** A circular scheduler queue used in event-driven simulators to track gate updates.
- **Switch-Level:** Transistor-level logic simulation where nodes represent strengths and states.
- **Prime Implicant:** A product term (cube) that cannot be combined with any other term to form a simpler term.
- **EPI:** A prime implicant that covers at least one minterm not covered by any other prime implicant.
- **Espresso:** An industrial heuristic logic minimizer that runs in polynomial average time.
- **Cofactor:** The Boolean function obtained by setting a specific variable to 0 or 1.
- **ROBDD:** A canonical directed acyclic graph representing a Boolean function, minimized using reduction rules.
- **Canonical Form:** A unique representation of a mathematical object, allowing easy comparison.

Frequently Asked / Viva Questions

Q1. Explain the advantage of event-driven simulation over compiler-driven simulation.

Compiler-driven simulation evaluates every gate in the circuit during every time step, which is slow. Event-driven simulation only evaluates gates whose inputs undergo a transition (an event). Since only 2 – 10% of gates change states in a clock cycle, event-driven simulation is 10 times faster.

Q2. Why are multiple strengths used in switch-level simulators?

Transistors can have different dimensions, driving strengths, or functions (e.g.

Frequently Asked / Viva Questions (contd.)

pull-up vs. pull-down). Using strengths allows the simulator to resolve contentions when multiple transistors drive the same bus line, ensuring that the stronger driver dominates.

Q3. What is the difference between Quine-McCluskey (QM) and Espresso?

QM is an exact minimization algorithm that guarantees the absolute minimum SOP form but has exponential time complexity. Espresso is a heuristic optimizer that uses Expand, Reduce, and Irredundant steps to find a near-optimal solution in polynomial average time, making it suitable for large circuits.

Q4. Why is variable ordering critical in ROBDDs?

The size (number of nodes) of an ROBDD is highly sensitive to the chosen variable order. A good ordering can result in a linear number of nodes ($O(n)$), while a poor ordering can make the node count grow exponentially ($O(2^n)$) for the exact same function.

Q5. What are cofactors, and how are they used in logic synthesis?

A cofactor is a simplified Boolean function obtained by setting a variable to a constant value ($f_{x_i} = f(x_i = 1)$). They are used in Shannon's expansion to recursively build BDDs and simplify logic expressions.

Q6. How does the Espresso algorithm use the "Reduce" operation to escape local minima?

The Reduce operation shrinks the size of prime cubes. This temporarily exposes some minterms, allowing the subsequent Expand step to expand cubes in different directions. This cycle helps the algorithm escape local minima.

Q7. What is a canonical representation, and why are ROBDDs canonical?

A canonical representation is a unique way of writing a mathematical object. ROBDDs are canonical because, for a fixed variable ordering, any Boolean function has exactly one unique ROBDD structure. This allows checking if two functions are identical in $O(1)$ constant time.

Q8. Explain the "Merge Duplicate Leaves" reduction rule in BDDs.

This rule combines all leaf nodes containing 0 into a single 0-terminal, and all leaves containing 1 into a single 1-terminal. This reduces the node count and creates a single converging point for all paths.

Q9. What is the function of the event queue in an event-driven simulator?

The event queue stores pending signal changes sorted by their execution times. As the simulator moves forward, it fetches events from the current time slot, evaluates the affected gates, and schedules new events at future slots.

Q10. Why do we need logic synthesis tools?

Logic synthesis automatically translates high-level functional code (Verilog/VHDL) into an optimized gate netlist. This speeds up design, guarantees logical correctness, and optimizes the gate count to meet timing and area constraints.

Comparison Snapshot**Gate-Level vs. Switch-Level Simulation:**

Feature	Gate-Level Simulator	Switch-Level Simulator
Primitive Element	Boolean logic gates (AND, OR, XOR).	MOS transistors (nodes, switches).
Execution Speed	Very fast (evaluates only on logic changes).	Slow (calculates charge flows and transistor strengths).
Accuracy	Ideal logic behavior.	Captures bi-directional pass transistor behavior.

BDD vs. ROBDD:

Feature	Binary Decision Diagram (BDD)	Reduced Ordered BDD (ROBDD)
Ordering	No fixed order of variables along paths.	Strict, identical variable ordering along all paths.
Uniqueness	Not unique (multiple representations).	Canonical (unique representation).
Leaf Nodes	Contains duplicate 0 and 1 terminal leaves.	Contains exactly one 0 and one 1 terminal.